

PalmQ10 Technical Guide

Architecture and internals, for maintainers — v0.1.3-alpha

19 July 2026

Contents

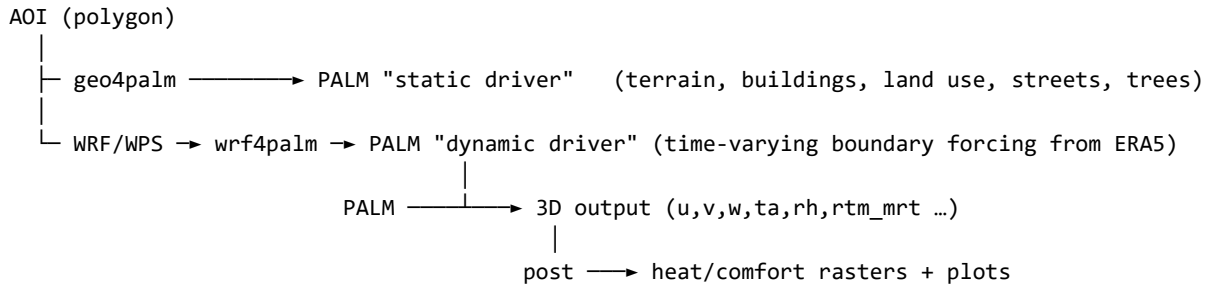
1. What PalmQ10 is	1
2. Repository layout	2
3. The central design idea: the base namelist	2
4. The pipeline, stage by stage	3
5. Execution model	4
6. Cross-cutting patterns & conventions	5
7. HPC subsystem — <code>internal/clusters/</code>	5
8. Containers — host-side vs container-side	6
9. Domain, grid & vertical datum notes	6
10. Known failure modes & how to diagnose (hard-won)	6
11. Making changes safely	7
12. Glossary	8

Audience. Developers who will maintain or extend `palmq10_core`. This explains *how the system is built and why*, not how to click through the GUI — see the *User Guide* for that. Read this end-to-end once; then use it as a reference.

1. What PalmQ10 is

PalmQ10 produces **urban-microclimate simulations** for a user-drawn area of interest (AOI). It orchestrates a chain of established atmospheric-science tools and turns their raw outputs into human-readable heat/comfort products (apparent temperature, MRT/UTCI, exceedance hours, etc.).

The scientific chain, in order:



- **WRF/WPS** — mesoscale weather model (coarse, ~1 km) that ingests **ERA5** reanalysis and produces the atmospheric state around the city.

- **wrf4palm** — interpolates WRF output onto the PALM grid to make the **dynamic driver** (the time-dependent lateral/top boundary conditions PALM needs).
- **geo4palm** — builds the **static driver** (the fixed geography: DEM terrain, building footprints/heights, land use, pavement, streets, tree canopy).
- **PALM** — the large-eddy-simulation core that actually resolves the urban microclimate.
- **post** — converts PALM's 3D NetCDF into biometeorological products.

Everything runs inside **Apptainer/Singularity containers** (.sif), either locally or on an HPC cluster. PalmQ10's own Python code is the *orchestration and glue* around those containers.

2. Repository layout

palmq10_plugin/	QGIS plugin – the GUI frontend
gui/aoi_dialog.py	main dialog; every pipeline button lives here
gui/cluster_profile_dialog.py	HPC profile editor
gui/process_dialog.py	live-log run window (QProcess)
schema/aoi_schema.py	AOI_SCHEMA – declares every user-editable field
palmq10_core/	
internal/	Backend pipeline (all the real logic; pure Python, no QGIS deps)
setup/	AOI → base namelist
geo4palm/	static driver
wrfwrs/	WRF/WPS (weather)
wrf4palm/	dynamic driver
palm/	PALM run + p3d generation
post/	products & plots
clusters/	HPC subsystem (schedulers, storage, auth)
util/	shared helpers
external/	Container images (.sif) + WPS_GEOG data (large, not in git)
jobs/	Per-project run data, created at runtime
documentation/	Cluster profiles (YAML), specs, this manual

Two-tree rule for maintainers. The **development** repo (this tree, .../Claude/Projects/PalmQ10/) holds palmq10_core/ and palmq10_plugin/. Production is a separate copy (currently an OneDrive tree). **Edit only the dev repo; the production copy is updated by a manual file copy.** Reading production to compare is fine; never edit it.

3. The central design idea: the base namelist

There is **one config artifact** that every stage reads:

jobs/{project}/setup/OUTPUT/base_namelist_{project}.txt

It is a plain-text **key = value # comment** file (not a Fortran namelist despite the name). It is the **single source of truth** for domain extent, grid (nx/ny/nz, dx/dy/dz), vertical grid, timing/spin-up, physics, cores, cluster profile, output selections, and CRS.

Config flow (know this cold — most “why is X wrong” bugs live here)

```
GUI form (aoi_schema.py)
| writes columns into
▼
```

```

GeoPackage jobs/{project}/setup/INPUT/{project}.gpkg    (one feature = the AOI, EPSG:4326)
| read by
▼
read_aoi_inputs() (setup/modules/aoi_reader.py) → frozen AOIInputs dataclass
| consumed by
▼
setup_base.py → writes base_namelist_{project}.txt
| read by every downstream stage via a base_namelist reader
▼
geo4palm / wrfwrs / wrf4palm / palm / post

```

A field that is not read at *every hop* silently vanishes. The classic bug: a GUI field exists in `aoi_schema.py` and lands in the GPKG, but `aoi_reader.py` never reads it, so `setup_base.py` falls back to a hard-coded default. When adding a new user setting you must touch: `aoi_schema.py` (declare) → `aoi_reader.py` (read into `AOIInputs`) → `setup_base.py` (write into the namelist) → the consuming stage. Miss any link and it's a phantom control.

4. The pipeline, stage by stage

Each stage has a `setup_*` step (host-side Python that prepares inputs/namelists) and often a `run_*` step (launches the container). Buttons in `aoi_dialog.py` trigger these as **external Python subprocesses** (env sanitized of QGIS PYTHON*/GDAL* vars).

4.1 setup — internal/setup/

- **setup_base.py** — the orchestrator. Reads the AOI GPKG, selects CRS (`modules/crs_select.py`), computes extents (`geom_extents.py`), the PALM grid (`palm_grid.py`, Temperton-friendly nx/ny), the vertical grid (`vertical_grid.py`), and the run window / spin-up (`datetime_utils.py`), then writes the base namelist (`modules/namelist_writer.py`).
- **modules/aoi_reader.py** — `read_aoi_inputs()` → `AOIInputs`. **The gatekeeper:** only fields read here reach the namelist.
- **create_folder_tree.py** — lays down `jobs/{project}/{setup,geo4palm,wrf-wrs,wrf4palm,palm,post}/...`

4.2 geo4palm (static driver) — internal/geo4palm/

- **setup_geo4palm.py** + **modules/api_call.py** — write geo4palm's config and fetch source data. Host-side downloaders: `get_dem_fabdem.py` / `get_dem_nasa.py` / `get_dem_esa.py` (DEM), `meta_chm_prepare.py` (Meta canopy height, AWS S3), `bld_raster_prepare.py` (buildings — reads a **prebuilt local GPKG**, no download), `flat_dem_prepare.py`, `get_sst_harmony.py` (stub).
- **run_geo4palm.py** — runs the geo4palm `.sif`. The container itself fetches **ESA WorldCover** land use and **OSM** streets/buildings (these are *container-side*, see §8).
- **Output:** `jobs/{project}/geo4palm/OUTPUT/{project}_static_N01` (NetCDF, no extension) — the PALM static driver.
- **api_call.py** also sets `z_origin = min(DEM)` in the config + base namelist so vertical datums line up with the wrf4palm atmosphere (see §9 vertical datum).

4.3 WRF/WPS (weather) — internal/wrfwrs/

- **setup_wrfwps.py** — computes the WRF domain (`modules/domain_plan.py`, `extent_bbox.py`), downloads **ERA5** (`modules/era5_cds.py` via CDS API, `cds_auth.py` for credentials), and writes `namelist.wps` + `namelist.input` (`modules/namelist_wps.py`, `namelist_input.py`). WRF core count auto-selected from grid size (target ≥ 1500 points/core, capped to avoid oversplitting).
- **run_wrfwps_apptainer.py** — runs `geogrid` → `ungrib` → `metgrid` → `real` → `wrf` in the `wrfwps.sif`. The exact container bash is generated by `modules/wrfwps_command.py`.

- **GEOG data:** geogrid needs the **WPS_GEOG** dataset. Locally it is bind-mounted from `local_wps_geog_dir`; on HPC it is the pre-staged `remote_geog_dir` (see §7 assets).
- **Output:** `met_em*` → `wrfout_d01_*` files (the raw weather the dynamic driver is built from).

4.4 wrf4palm (dynamic driver) — `internal/wrf4palm/`

- `setup_wrf4palm.py` + `modules/namelist_writer.py` — write `namelist.wrf4palm`.
- `run_wrf4palm.py` — interpolates `wrfout` onto the PALM grid → the **dynamic driver**. On HPC it first does a **pre-flight scratch check** (are the WRF `wrfout` files still on scratch? stage symlinks, upload only the namelist) *before* `run_hpc_job` — this is why it establishes cluster auth up front (see §7 auth).
- **Output:** `{project}_dynamic` driver.

4.5 PALM — `internal/palm/`

- `palm_setup.py` — stages the static + dynamic drivers into PALM's **INPUT/**, then generates the **p3d** parameter file (`modules/write_p3d.py`) from the base namelist (grid, bc, physics, `netcdf_deflate`, `RTM/localized_raytracing`, `npex/npey`, output list).
- `run_palm_apptainer.py` — runs PALM (`modules/palm_command.py` builds the container command; `palm_local_apptainer.py` for the local path). Live progress via `modules/monitoring.py`.
- **Output:** `jobs/{project}/palm/OUTPUT/{project}_3d.000.nc` (the 3D result).

4.6 post — `internal/post/`

- `run_post.py` — orchestrates product generation. Reads the timezone from the base namelist.
- `modules/compute_products_hor_3d.py` / `compute_products_ver_3d.py` — horizontal & vertical slice products (apparent temp, MRT, UTCI, wind, exceedance). Level selection is `origin_z`-aware.
- `modules/exceedance_plots_*.py`, `export_outputs.py`, `smoothen_*` — plots and exports.

5. Execution model

5.1 GUI button → subprocess

`aoi_dialog.py` never imports pipeline code directly (QGIS Python != the pipeline's Python environment). Each button shells out: it builds a command and runs a `run_*/setup_*` script as an external process, streaming logs to a window (`process_dialog.py`, `QProcess`) or a spawned terminal.

5.2 Windows → WSL relaunch (critical)

Cluster ops need Linux `ssh/rsync/ssh-agent`, which native Windows lacks. So **every HPC-capable entry point calls `relaunch_in_wsl_if_needed()` first**: on `os.name == "nt"` it re-execs itself via `wsl.exe bash -lc "python3 ..."` and exits. Consequences you must remember: - Cluster `ssh/rsync`, the credential vault, the `ssh-agent`, and `cryptography` all live in **WSL**, not Windows and not QGIS Python. - Paths cross the Windows↔WSL boundary: `0:\foo` ↔ `/mnt/o/foo`. Helpers convert these; watch for it.

5.3 Local vs HPC backend

`heavy_calcs_exe_backend` in the base namelist decides: - **local** — run the `.sif` directly on the workstation via Apptainer. - **hpc** — go through the cluster subsystem (§7). The `cluster_profile` field names the YAML.

5.4 Root detection (all scripts)

1. `PALMQ10_ROOT` env var if valid; 2. search ancestors for a dir containing `jobs/` + `internal/`;
2. fall back to `~/Documents/palmq10`.

6. Cross-cutting patterns & conventions

- **Network-download resilience.** API/HTTP downloads (ERA5 CDS, OpenTopography, geo4palm sources, Meta CHM S3) retry **indefinitely** with an elapsed-time heartbeat and a “Ctrl-C to abort” hint. File validation is **structural, not size-based** (e.g. ERA5 checks the GRIB GRIB...7777 envelope, not a byte threshold — a valid small-domain GRIB is only a few kB). See `era5_cds.py::_is_usable_file`.
- **Cluster transfer resilience.** rsync uploads/downloads use `--partial` + bounded retry. `upload_dir` also uses `--delete` (mirror) — safe only when the local source is the complete authoritative copy.
- **Container command generation.** The bash that runs inside each `.sif` is generated by a `*_command.py` module (e.g. `wrfwps_command.py`, `palm_command.py`) as an f-string. Edit MPI flags, `ulimit`, `env` exports there — **not** in the container.
- **base_namelist readers.** Each stage has its own small parser (`*/modules/base_namelist*.py`). They read the same file but expose stage-relevant fields.
- **Show-before-apply.** When changing generated artifacts (namelists, job scripts), verify the produced output, not just that the code compiles.

7. HPC subsystem — `internal/clusters/`

7.1 Adapter pattern

Three orthogonal adapters compose a cluster: - **scheduler** — `scheduler_adapters/slurm_adapter.py` (Sirius), `sge_adapter.py` (Rigel): `submit/poll/cancel` + status parsing. - **storage** — `storage_adapters/lustre_adapter.py` (and `scratch_lustre_adapter.py`): `ssh_run`, `upload_file/dir`, `download_dir`, `ensure_remote_dir`, `rsync` + SSH ControlMaster. - **runtime** — `runtime_adapters/apptainer_adapter.py`: build the apptainer `exec` ... invocation.

7.2 Orchestrator — `hpc_runner.py`

`run_hpc_job(HPCJobRequest(...))` is the single entry. Lifecycle: 1. `ensure_cluster_auth(profile, root)` (vault, §7.4) — unless the caller already did it. 2. Create remote workspace, clear old monitoring, upload inputs (unless `skip_input_upload`). 3. Render + upload the scheduler script (from the stage’s `*_command.py`), submit. 4. Poll status; every `monitoring_sync_seconds` download the MONITORING folder (live logs). 5. On finish: download OUTPUT; on **FAILED**, print the tail of `*.err` + `geogrid/ungrib/metgrid/ real/wrf` logs inline (so you don’t have to SSH), then raise. 6. finally: `teardown_cluster_auth`.

7.3 Cluster profiles — `documentation/cluster_profiles/*.yaml`

Declare `cluster_name`, `username`, `login_host`, `ssh_port`, scheduler/storage/runtime types, `remote_scratch_root`, `remote_geog_dir`, `partition/queue`, `auth_mode`, `mem/core` caps, SSH multiplexing (`ssh_control_path`, `ssh_control_persist`). `profile_reader.py` loads them. - **Rigel** (SGE, DELL queue, `max_single_node_slots`: 36, `node_memory_gb`: 187, `default_walltime`: 50:00:00). **Sirius** (SLURM, gen partition, `max_single_node_slots`: 256, `node_memory_gb`: 512, `default_walltime`: 60:00:00; SSH username has a literal @: `aramcue@upvnet.upv.es@sirius.upv.es`) - Values above are the **shipped production profiles** as of v0.1.3-alpha. Both walltimes are deliberately long — a job killed at its limit loses everything computed so far. Treat the profile as the source of truth rather than these notes; earlier revisions of this manual quoted a 112-slot Sirius and a 4-hour walltime, both since superseded.

7.4 Credential vault (key auth) — `credential_vault.py`

Kills the 4–5×-per-job SSH password prompts. Gated by `auth_mode`: `key_vault` in the profile; the code default is `password_interactive` (unchanged legacy flow), but **both shipped profiles now set `key_vault`**,

so it is on by default in practice. - A **passphrase-protected ed25519 key per (cluster, user)**; the passphrase is stored in a per-user **encrypted vault** (scrypt KDF + Fernet) at `~/.palmq10/vault.enc`. - One **master password** per job unlocks the vault → key loaded into a **per-job ephemeral ssh-agent** → all ssh/rsync in that job are silent (survives VPN drops). `teardown` kills the agent. - **ensure_cluster_auth** is **idempotent**: if `PALMQ10_KEY_AUTH_READY=1` it returns a **reused** context (no second master-password prompt). This lets a stage (e.g. `wrf4palm`) unlock **before** a pre-flight step and have `run_hpc_job` reuse the same session. - **Master-password GUI** (`master_password_setup.py` + `create_master_password` / `change_master_password` in `credential_vault.py`): create/rotate the master password from the settings dialog without touching the SSH keys.

7.5 Asset bootstrap — `bootstrap_cluster_assets.py`

Uploads the **container .sif images** and the **WPS_GEOG** dataset to the cluster once (they are too big to ship per-job). `--upload-geog` rsyncs the geog to `remote_geog_dir`; in HPC mode the run consumes that pre-staged copy (`local_wps_geog_dir` is ignored). `fix_geog_con` handles a geog `orogwd_10m/con` compatibility symlink.

8. Containers — host-side vs container-side

Four images live in `external/`: `geo4palm`, `wrfwps`, `wrf4palm`, `palm` (.sif). Their **source code is not in this repo** — it lives at `0:\Fulbright_2025\Containers\{geo4palm,palm,wrf4palm,wrfwps}\`. Rebuilding a .sif is the only way to change *container-side* code.

What runs where matters for debugging: - *Host-side* (PalmQ10 Python, editable here): all `setup_*`, all `run_*` orchestration, the DEM/CHM/building downloaders, ERA5 download, namelist generation, the generated container bash. - *Container-side* (sealed unless you rebuild the .sif): `geo4palm`'s ESA WorldCover + OSM downloads (`/opt/GE04PALM/util/`), the actual WRF/WPS/PALM binaries. Host-side retry wrappers cannot cover container-side download flakiness — that needs a rebuild or a bind override.

9. Domain, grid & vertical datum notes

- **PALM decomposition:** `cores = npex × npey`; each subdomain must be `≥ min_subdomain_points` (default 16). Prefer near-square factors. On these urban domains you are usually **hardware-core limited** (256 on Sirius, 36 on Rigel), so `min_subdomain_points` rarely binds — bigger subdomains = fewer cores = *more* memory per core (relevant to OOM on big/high-res domains).
- **Spin-up:** `spinup_hours` in the base namelist prepends spin-up to the analysis window. The WRF/PALM run length = `spinup_hours + analysis`. `0` gives a run equal to just the analysis window.
- **Vertical datum:** `geo4palm` sets `z_origin = min(DEM)` (via `api_call.py`) and PALM shifts terrain to the domain floor; keeping `z_origin` consistent across the static driver, base namelist, and `wrf4palm` avoids a ~200 m vertical mismatch between PALM's floor and the WRF atmosphere.

10. Known failure modes & how to diagnose (hard-won)

Symptom	Root cause	Fix / where
geogrid fails in ~1 s, <code>exit 22</code> , tiny MaxRSS; log says <code>Could not open .../<dataset>/index</code>	The cluster WPS_GEOG copy is missing a dataset's <code>index</code> file (incomplete/interrupted upload; <code>index</code> sorts <i>after</i> the numeric tiles so it's the first casualty of a truncated transfer). NOT memory/stack.	Re- <code>rsync</code> the geog from a complete local copy. Verify with the recursive “tiles-but-no-index” scan.
ERA5 download retries forever, “file too small”	Size-threshold validation rejecting a valid <i>small-domain</i> GRIB (a few kB).	Structural GRIB validation (<code>GRIB...7777</code>) in <code>era5_cds.py</code> .
geo4palm static build fails intermittently on ESA WorldCover	Server-side Terrascope flakiness (container-side).	Retry (host-side wrapper retries the whole container run).
wrf4palm write SIGSEGV during NetCDF write	Oversized PALM domain → multi-GB driver → <code>to_netcdf</code> segfault.	Right-size the AOI; combined write is fine for normal domains.
PALM blow-up <code>PAC0318 / -9999</code> top forcing	Bad top-boundary config.	Fixed via config, not container edits.
PALM <code>PAC0337/PCM0010/RAD0051</code> at init	Vertical stretch onset below the buildings.	Grid config (uniform, or stretch above tallest roof + <code>z_origin = min(DEM)</code>).
post UTCI all-NaN (<code>All-NaN slice</code>)	Level pick ignored <code>origin_z</code> ; MRT sampled at wrong level.	<code>compute_products_hor_3d.py</code> <code>origin_z</code> -aware level pick + near-surface MRT.
New GUI control does nothing	Field not read in <code>aoi_reader.py</code> (phantom control).	Wire it through all four hops (§3).
HPC job “FAILED” with no visible error	Local <code>.out</code> was a stale early monitoring snapshot.	<code>run_hpc_job</code> now refreshes <code>.out</code> and prints <code>.err</code> /log tails on FAILED.
Instant geogrid/real segfault, no output, on one cluster only	Small stack <code>ulimit</code> on that cluster.	<code>ulimit -s unlimited</code> in the generated container bash (<code>wrfwps_command.py</code>).

First move for any HPC failure: `sacct -j <id> --format=JobID,State,ExitCode,MaxRSS,ReqMem,Elapsed,Reason%50` (SLURM) — it classifies OOM vs signal vs exit code instantly. Then read `*.err` + the stage log (`geogrid.log`, `rs1.error.0000`, ...) on scratch.

11. Making changes safely

1. Edit the **dev repo** only (`palmq10_core/` + `palmq10_plugin/`). Production is copied manually.
2. For a new user setting, wire all four hops (§3).
3. For container behavior, edit the generating `*_command.py` (host-side) if possible; a real container change needs a `.sif` rebuild from the 0: source tree.
4. Syntax-check Python via WSL (`python3 -c "import ast; ast.parse(open(f).read())"`); for generated namelists/scripts, inspect the produced file.
5. Prefer bounded retry for cluster ops (a failure is usually real auth/disk), indefinite retry for flaky public download endpoints.

12. Glossary

- **WPS / geogrid / ungrib / metgrid / real** — WRF Preprocessing System stages: interpolate static geography (geogrid), decode GRIB (ungrib), merge onto the model grid (metgrid), build WRF initial/boundary files (real).
- **WPS_GEOG** — the large static geography dataset WPS/geogrid reads; each dataset folder needs an index metadata file.
- **static driver** — PALM's fixed geography NetCDF (from geo4palm).
- **dynamic driver** — PALM's time-varying boundary forcing NetCDF (from wrf4palm).
- **p3d** — PALM's parameter/namelist file (`write_p3d.py`).
- **RTM** — PALM's Radiative Transfer Model (urban radiation); `localized_raytracing`.
- **MRT / UTCI** — mean radiant temperature / Universal Thermal Climate Index (comfort metrics).
- **ERA5** — ECMWF reanalysis; the weather source, downloaded via the CDS API.
- **Apptainer/Singularity .sif** — the container image format PalmQ10 runs everything in.
- **Lustre / scratch** — the HPC parallel filesystem; scratch may auto-purge un-accessed files.

Keep this manual current: when you change a stage's contract, the config flow, or the HPC lifecycle, update the relevant section. It is the fastest path to onboarding the next maintainer.